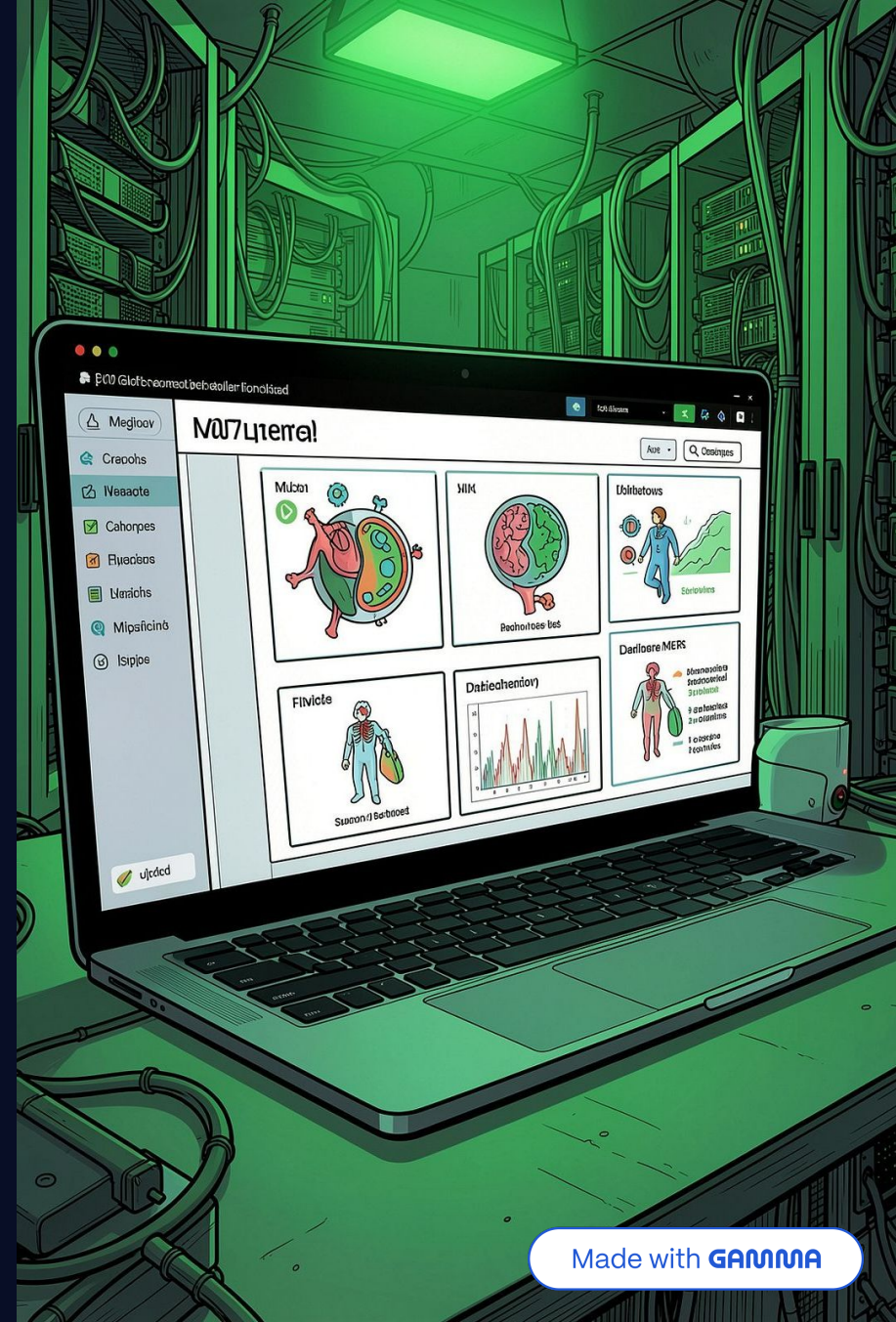


Mere Medical

By Ali Ateeq

Mere Medical is a virtualized healthcare application deployed on an Ubuntu Server inside Proxmox. The project uses Docker, Containerlab, WordPress, and OAuth2 Proxy to provide containerized services, virtual networking, secure user authentication, and access to demo medical data.

DEVOPS · SECURITY · HEALTHCARE



Made with GAMMA

Infrastructure Overview



Host Layer

Proxmox VE – 192.168.1.41:8006

Guest VM

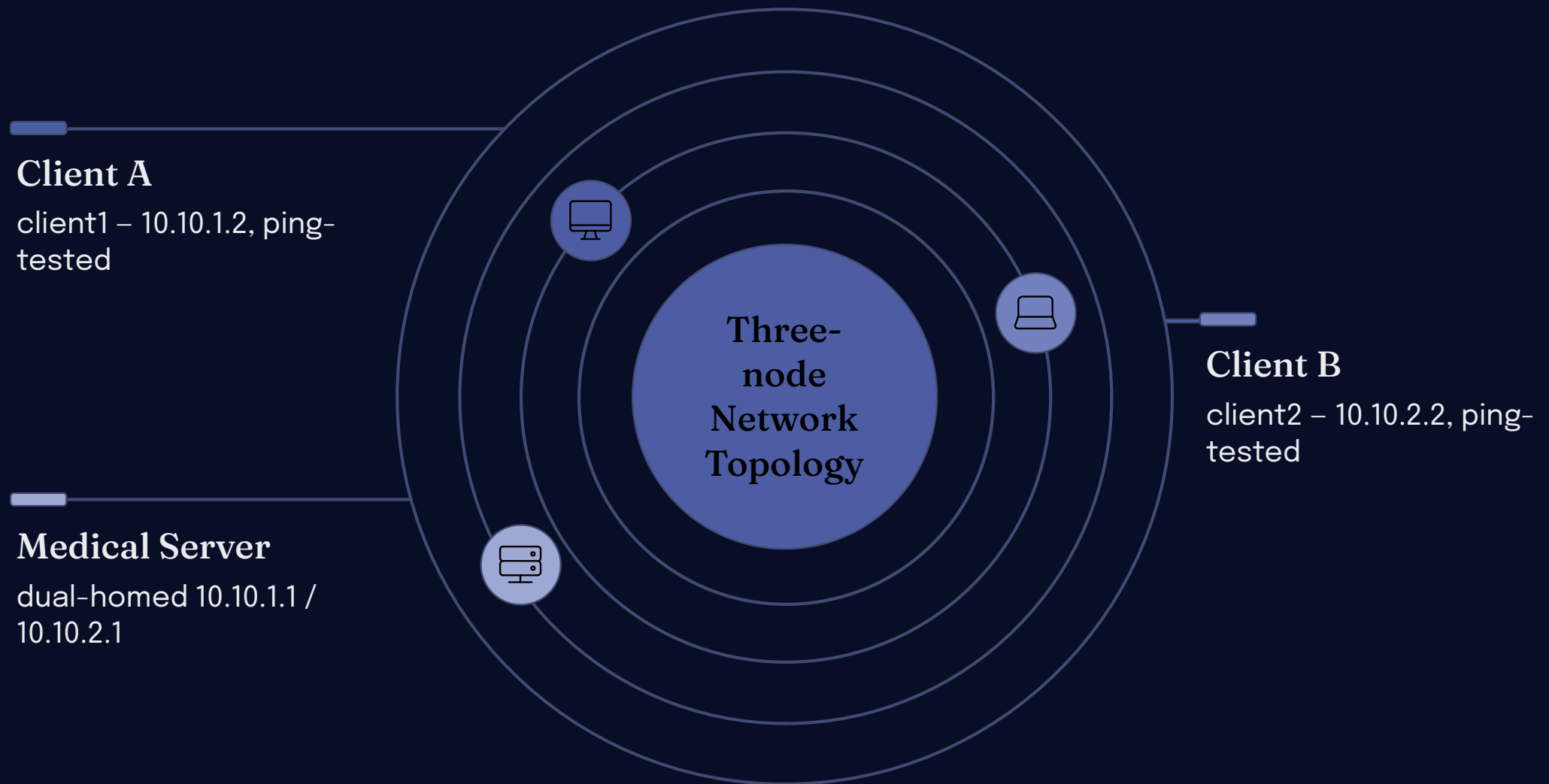
Ubuntu Server – 192.168.1.66

Managed via **SSH** from Windows

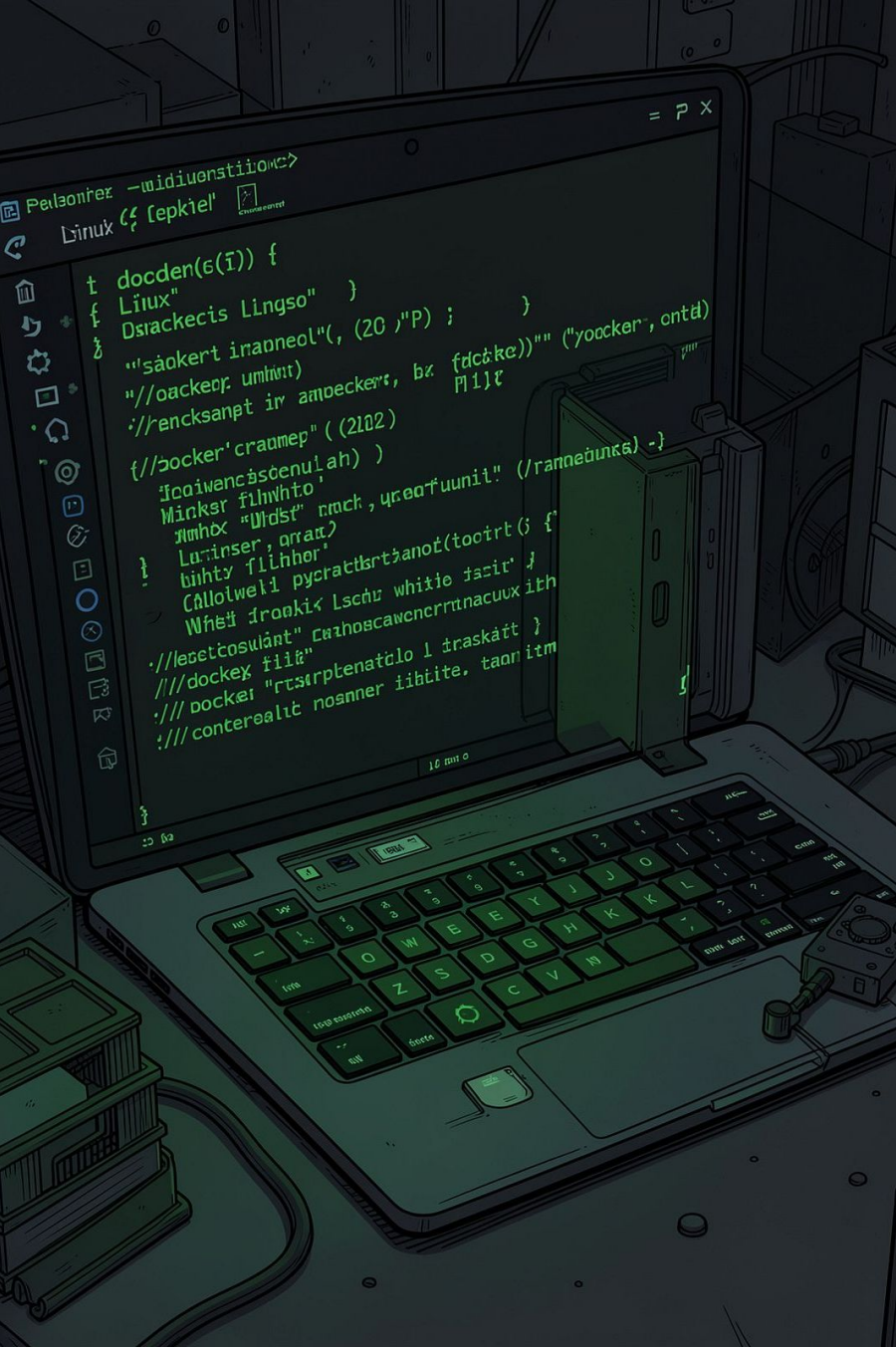
Core Tools

- Docker & Docker Compose
- Containerlab
- Python

Containerlab Network Topology



Two isolated client subnets converge on a dual-homed medical server. Connectivity validated via ping across all interfaces.



Containerized Services

mere-medical

App image: `cfu288/mere-medical:latest`

WordPress

OAuth / OIDC identity provider

MySQL

Persistent WordPress database

OAuth2 Proxy

Reverse proxy enforcing auth

WordPress as OIDC Provider

miniOrange OAuth Server Plugin

WordPress becomes a full OpenID Connect issuer for Mere Medical.

Application Registration

Name: Mere Medical Login

Flow: Authorization Code

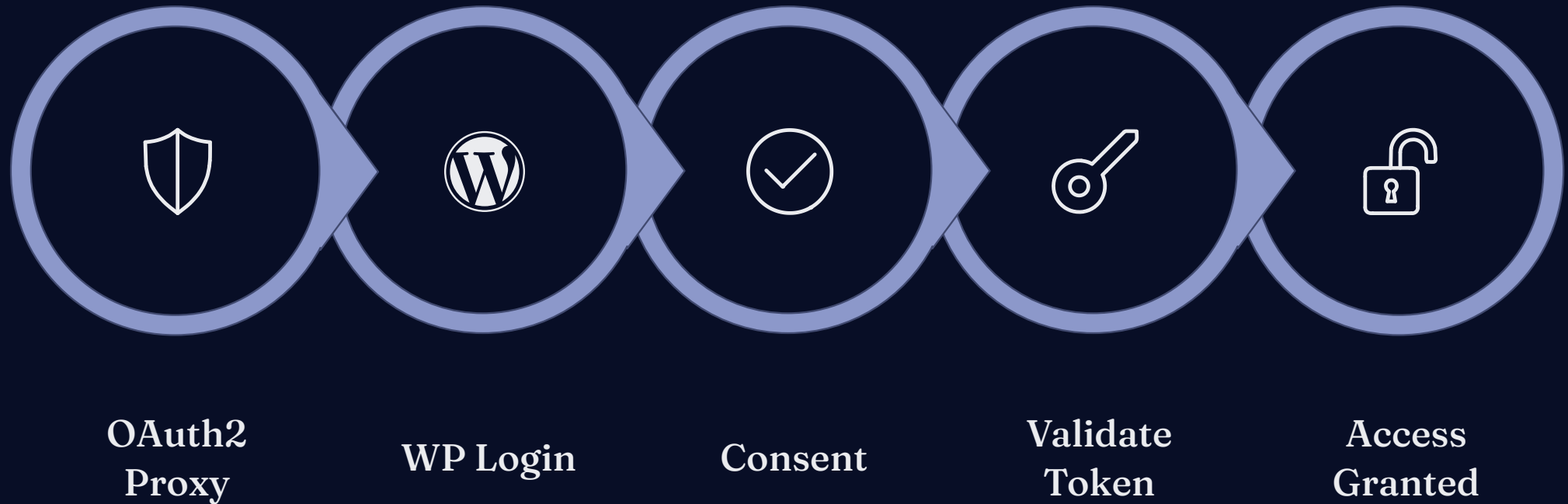
Scopes: openid · profile · email

Secrets Management

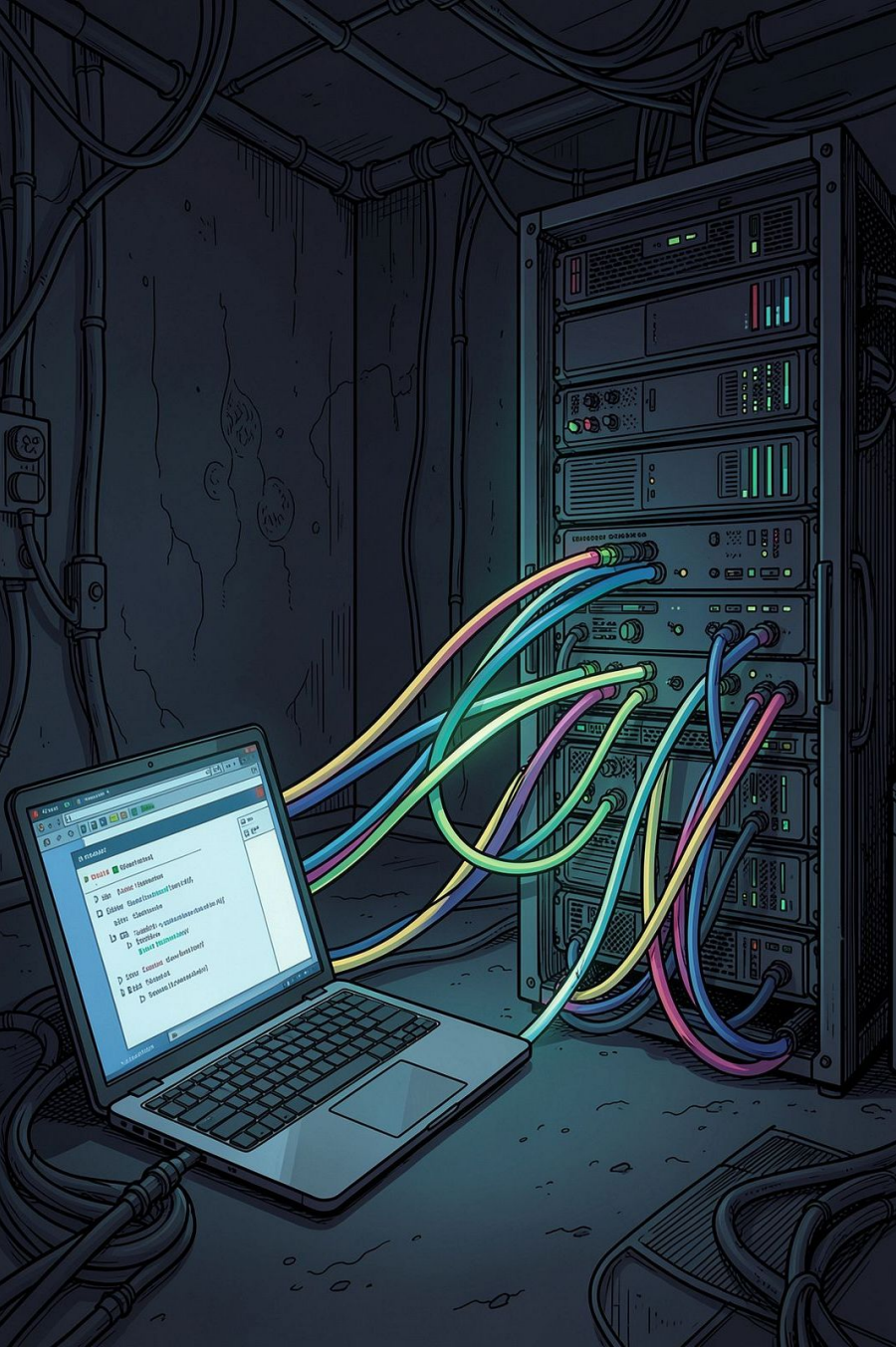
Client ID, Secret, Issuer URL, Cookie Secret → stored in protected `.env`



Authentication Flow



Direct access to Mere Medical is **blocked**. Every request must pass through OAuth2 Proxy → WordPress OIDC → consent approval.



SSH Tunnel — Local Access

Tunnel Command

```
ssh -N -L 4300:127.0.0.1:4200 ali@192.168.1.66
```

Access URL

```
http://localhost:4300
```

Why a Tunnel?

Mere Medical binds to localhost inside the VM – SSH forwards the port to the Windows host.

Demo Data — Patient Records



Source File

`/app/web/assets/demo.json`

Data Collections

- Clinical records
- Connections
- Users
- Preferences

Persistence Strategy

Copied out → edited to include **20 demo patients** → mounted back as a volume. Survives container recreation.

Operations & Troubleshooting



Start / Stop

Compose up/down for all services; VM reboot via Proxmox



YAML Errors

Indentation fixes in `docker-compose.yml` and Containerlab topology



Network Tests

ping between client1, client2, and medical-server subnets



Password Recovery

Proxmox and Ubuntu password reset procedures documented

Project Startup Commands

Follow these steps to initialize and access the Mere Medical environment, ensuring all services are running and accessible.

Step	Environment	Purpose	Command
1	Browser	Open Proxmox	<code>https://192.168.1.41:8006</code>
2	PowerShell	Connect to Ubuntu	<code>ssh ali@192.168.1.66</code>
3	Ubuntu	Open Containerlab directory	<code>cd ~/mere-medical-lab</code>
4	Ubuntu	Deploy Containerlab topology	<code>containerlab deploy -t lab.clab.yml</code>
5	Ubuntu	Start existing lab containers	<code>docker start clab-meremedical-client1 clab-meremedical-client2 clab-meremedical-medical-server</code>
6	Ubuntu	Start WordPress and MySQL	<code>cd ~/wordpress-app && docker compose up -d</code>
7	Ubuntu	Start Mere Medical and OAuth2 Proxy	<code>cd ~/mere-medical-app && docker compose up -d</code>
8	Ubuntu	Display all running containers	<code>docker ps</code>
9	PowerShell	Create the SSH tunnel	<code>ssh -N -L 4300:127.0.0.1:4200 ali@192.168.1.66</code>
10	Browser	Open Mere Medical	<code>http://localhost:4300</code>

Main Project Links

Service	Address
Proxmox	https://192.168.1.41:8006
WordPress	http://192.168.1.66:8080
WordPress Admin	http://192.168.1.66:8080/wp-admin
Mere Medical	http://localhost:4300